

Dow Data Challenge: LARS Regression

CHE 891 — Week 2

We apply **Least Angle Regression (LARS)** to the Dow impurity dataset. LARS is well-suited here because:

- The 42 process variables are strongly collinear
- We want a **sparse** model — only the most relevant sensors
- LARS produces the full regularization path at the cost of one OLS fit

```
In [1]: import pandas as pd
import numpy as np
from pylab import *
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import TransformedTargetRegressor
from sklearn.linear_model import Lars, lars_path
import warnings
warnings.filterwarnings('ignore')
```

1. Load Data

```
In [2]: training_url = 'http://egr.msu.edu/~scb/courses/EGR891/dow_training_clean.csv'

df_train = pd.read_csv(training_url, parse_dates=['Date'])
print(f"Training: {df_train.shape}")
df_train.head()
```

Training: (1927, 48)

Out[2]:

	Date	x1:Primary Column Reflux Flow	x2:Primary Column Tails Flow	x3:Input to Primary Column Bed 3 Flow	x4:Input to Primary Column Bed 2 Flow	x5:Primary Column Feed Flow from Feed Column	x6:Primary Column Make Flow	x7:Primary Column Base Leve
0	2015-12-01 00:00:00	327.813	45.7920	2095.06	2156.01	98.5005	95.4674	54.3476
1	2015-12-01 09:00:00	327.686	45.8192	2109.75	2185.82	98.8448	95.5414	54.0806
2	2015-12-01 14:00:00	322.439	46.0614	2108.64	2124.04	96.8568	94.1683	54.5115
3	2015-12-01 15:00:00	322.402	45.9821	2104.28	2233.15	96.8940	94.1798	54.2862
4	2015-12-01 20:00:00	330.082	45.9936	2104.54	2171.53	98.4124	96.2888	55.2908

5 rows × 48 columns



2. Prepare X and y

```
In [3]: feat_cols = [c for c in df_train.columns if c.startswith('x')]

X_train      = df_train[feat_cols].values
y_train      = df_train['y:Impurity'].values

print(f"Features: {len(feat_cols)}")
print(f"X_train: {X_train.shape} y_train: {y_train.shape}")
```

Features: 42

X_train: (1927, 42) y_train: (1927,)

3. Pipe and Model

We use a `Pipeline` (scaling inside) and `TransformedTargetRegressor` (softplus transform on y).

The **softplus inverse** $z = \ln(e^y - 1)$ maps positive y to the real line before fitting. Predictions are automatically inverse-transformed back via softplus $y = \ln(1 + e^z)$.

```
In [4]: def softplus(z):
        return np.logaddexp(z, 0)      # log(1 + exp(z)), numerically stable

def softplus_inv(y):
    return np.log(np.expm1(y))        # log(exp(y) - 1), valid for y > 0

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('lars', Lars(n_nonzero_coefs=len(feats_cols)))
])

model = TransformedTargetRegressor(
    regressor=pipe,
    func=softplus_inv,      # y → z for training
    inverse_func=softplus   # z → y for prediction
)
model.fit(X_train, y_train)
print("Model fitted.")
```

Model fitted.

4. LARS Path

`lars_path` computes the full regularization path directly on scaled X and transformed y . It returns `(alphas, active, coefs)` where `coefs` has shape `(n_features, n_steps)`.

```
In [5]: X_s = model.regressor_['scaler'].transform(X_train)
        z_train = softplus_inv(y_train)

# max_iter=len(feats_cols) stops at the OLS solution (same as Lars(n_nonzero_coefs=p
_, _, path = lars_path(X_s, z_train, method='lar', max_iter=len(feats_cols))
print(f"Steps in path: {path.shape[1]}")
print(f"Features:      {path.shape[0]}")
```

Steps in path: 43
Features: 42

5. LARS Path Plot

Each line is one sensor variable. Variables enter the active set one at a time as the L1 norm of the coefficient vector grows (left = sparse, right = OLS).

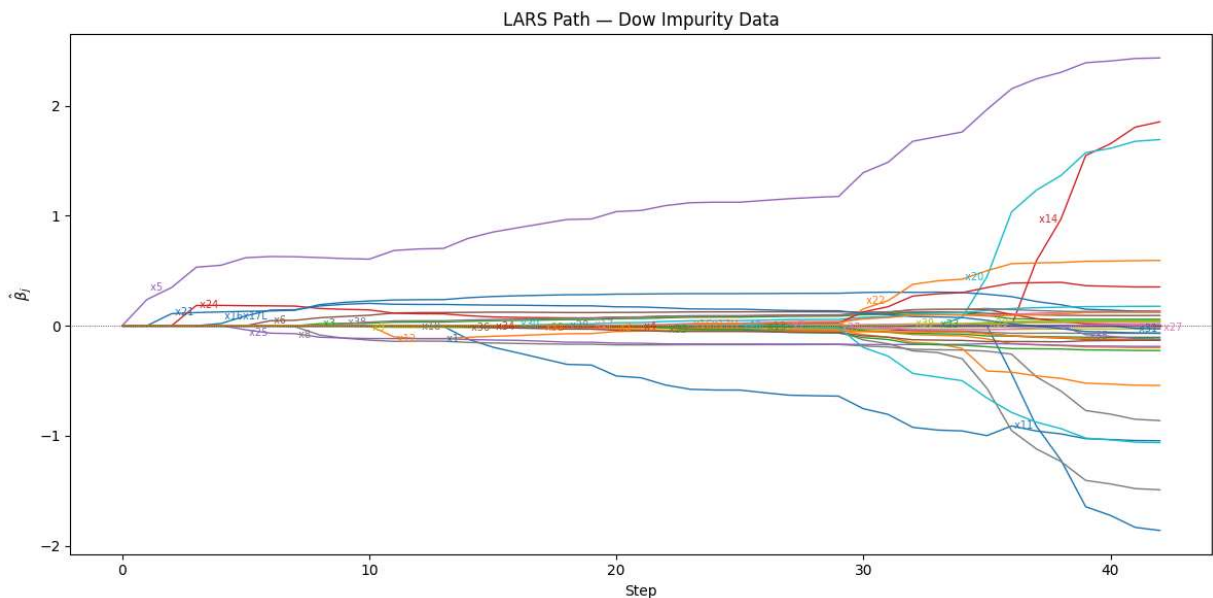
```
In [6]: short_names = [c.split(':')[0] for c in feats_cols]
        steps = np.arange(path.shape[1])

figure(figsize=(12, 6))
line_colors = [plot(steps, path[i, :], lw=1)[0].get_color()
               for i in range(len(short_names))]

axhline(0, color='k', lw=0.5, ls=':')
```

```
# Label each variable at its entry step
for i, name in enumerate(short_names):
    nz = np.where(path[i, :] != 0)[0]
    if len(nz) > 0:
        step = nz[0]
        y_pos = path[i, min(step + 1, path.shape[1] - 1)]
        text(step, y_pos, f' {name}',
             fontsize=7, color=line_colors[i], va='center')

xlabel('Step')
ylabel(r'$\hat{\beta}_j$')
title('LARS Path — Dow Impurity Data')
tight_layout()
show()
```



6. Variable Entry Order

The order in which variables enter the active set reflects their marginal correlation with the residual — a natural variable importance ranking.

```
In [7]: # Find the step at which each variable first becomes nonzero
entry_step = []
for i in range(path.shape[0]):
    nz = np.where(path[i, :] != 0)[0]
    entry_step.append(nz[0] if len(nz) > 0 else path.shape[1])

order = np.argsort(entry_step)

print("Variable entry order:")
for rank, idx in enumerate(order):
    if entry_step[idx] < path.shape[1]:
        print(f" {rank+1:2d}. {feat_cols[idx]}")
```

Variable entry order:

1. x5:Primary Column Feed Flow from Feed Column
2. x21:Primary Column Bed 1 Temperature
3. x24: Secondary Column Tails Flow
4. x16x17L
5. x25: Secondary Column Tray DP
6. x6:Primary Column Make Flow
7. x8:Primary Column Reflux Drum Pressure
8. x3:Input to Primary Column Bed 3 Flow
9. x38: Feed Column Calculated DP
10. x9:Primary Column Condenser Reflux Drum Level
11. x12:Primary Column Bed3 DP
12. x18:Primary Column Bed 4 Temperature
13. x1:Primary Column Reflux Flow
14. x36: Feed Column Recycle Flow
15. x34: Secondary Column Tails Temperature
16. x30: Secondary Column Bed 1 Temperature
17. x32: Secondary Column Tray 2 Temperature
18. x28: Secondary Column Base Temperature
19. x17:Primary Column Tails Temperature 1
20. x2:Primary Column Tails Flow
21. x4:Input to Primary Column Bed 2 Flow
22. x33: Secondary Column Tray 1 Temperature
23. x16x17H
24. x40: Feed Column Tails Flow
25. x13:Primary Column Bed4 DP
26. x7:Primary Column Base Level
27. x37: Feed Column Tails Flow to Primary Column
28. x22: Secondary Column Base Concentration
29. x39: Feed Column Steam Flow
30. x23: Flow from Input to Secondary Column
31. x20:Primary Column Bed 2 Temperature
32. x29: Secondary Column Tray 3 Temperature
33. x11:Primary Column Bed2 DP
34. x14:Primary Column Base Pressure
35. x35: Secondary Column Tails Concentration
36. x31: Secondary Column Bed 2 Temperature
37. x27: Secondary Column Base Pressure

7. C_p Model Selection

Mallows' C_p estimates prediction risk at each step of the LARS path:

$$C_p = \frac{\text{RSS}_p}{\hat{\sigma}^2} - n + 2p$$

where n is the number of training observations, p is the number of active variables plus intercept, and $\hat{\sigma}^2$ is estimated from the full OLS model (last step).

```
In [8]: n = len(z_train)
        z_bar = z_train.mean()

        # Predictions at each step: intercept = z_bar (X_s is zero-mean)
```

```

z_hat = z_bar + X_s @ path          # shape (n, n_steps)
rss = np.sum((z_train[:, None] - z_hat) ** 2, axis=0)

# Number of parameters at each step (nonzero coefficients + intercept)
p_s = (path != 0).sum(axis=0) + 1

# sigma^2 from OLS (last step)
sigma2 = rss[-1] / (n - p_s[-1])
print(f"sigma^2 = {sigma2:.4f}, sigma = {np.sqrt(sigma2):.4f}")

Cp = rss / sigma2 - n + 2 * p_s

best_step = np.argmin(Cp)
print(f"Optimal step: {best_step} ({int(p_s[best_step]) - 1} variables, Cp = {Cp[

```

```

sigma^2 = 0.4027, sigma = 0.6345
Optimal step: 31 (28 variables, Cp = 11.2)

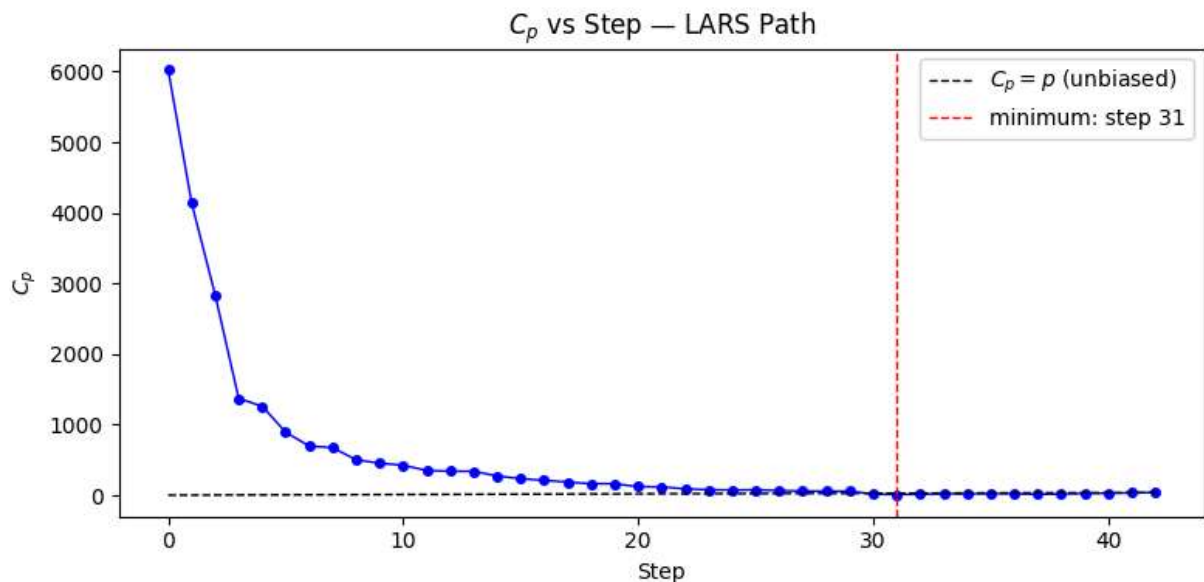
```

```

In [9]: steps = np.arange(path.shape[1])

figure(figsize=(8, 4))
plot(steps, Cp, 'b-o', ms=4, lw=1)
plot(steps, p_s, 'k--', lw=1, label=r'$C_p = p$ (unbiased)')
axvline(best_step, color='r', ls='--', lw=1, label=f'minimum: step {best_step}')
xlabel('Step')
ylabel(r'$C_p$')
title(r'$C_p$ vs Step - LARS Path')
legend()
tight_layout()
show()

```



8. Training and Validation Fit

Refit the model using only the `best_step` variables selected by C_p , then evaluate R^2 on both the training and validation sets.

```
In [10]: # Load validation data
validation_url = 'http://egr.msu.edu/~scb/courses/EGR891/dow_validation_clean.csv'
df_val = pd.read_csv(validation_url, parse_dates=['Date'])
print(f"Validation: {df_val.shape}")

X_val = df_val[feat_cols].values
y_val = df_val['y:Impurity'].values
```

Validation: (5891, 48)

```
In [11]: from sklearn.metrics import r2_score

# Re-compute training path
X_s_train = model.regressor_['scaler'].transform(X_train)
z_train = softplus_inv(y_train)
_, _, path_train = lars_path(X_s_train, z_train, method='lar', max_iter=len(feat_co

n_train = len(z_train)
z_bar_train = z_train.mean()
z_hat_train = z_bar_train + X_s_train @ path_train
rss_train = np.sum((z_train[:, None] - z_hat_train) ** 2, axis=0)
p_s = (path_train != 0).sum(axis=0) + 1
sigma2 = rss_train[-1] / (n_train - p_s[-1])
Cp = rss_train / sigma2 - n_train + 2 * p_s
best_step = np.argmin(Cp)

print(f"Best step from training: {best_step} ({int(p_s[best_step]) - 1} variables)")
```

Best step from training: 31 (28 variables)

```
In [12]: # Apply training model to validation data
X_s_val = model.regressor_['scaler'].transform(X_val)
z_val = softplus_inv(y_val)

# Predict using training coefficients at best_step
z_bar_train = z_train.mean()
coef_best = path_train[:, best_step]
z_pred_val = z_bar_train + X_s_val @ coef_best

# Transform back to original scale
y_pred_val = softplus(z_pred_val)

# Evaluate
r2_val = r2_score(y_val, y_pred_val)
r2_train = r2_score(y_train, softplus(z_bar_train + X_s_train @ coef_best))

print(f"Training R²: {r2_train:.4f}")
print(f"Validation R²: {r2_val:.4f}")
```

Training R²: 0.7747

Validation R²: 0.3378

```
In [13]: # Parity plot: predicted vs actual
figure(figsize=(10, 4))

subplot(1, 2, 1)
```

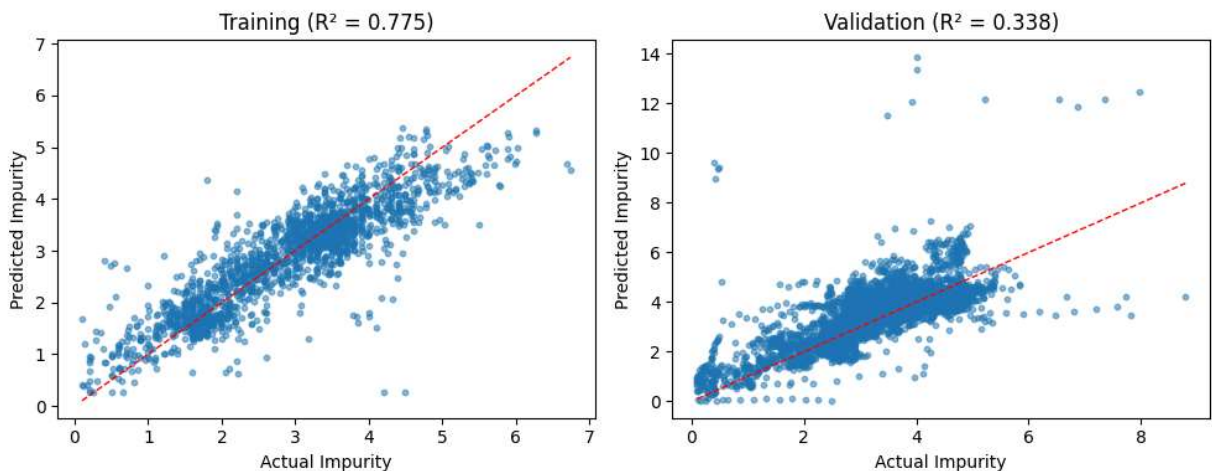
```

scatter(y_train, softplus(z_bar_train + X_s_train @ coef_best), alpha=0.5, s=10)
plot([y_train.min(), y_train.max()], [y_train.min(), y_train.max()], 'r--', lw=1)
xlabel('Actual Impurity')
ylabel('Predicted Impurity')
title(f'Training (R² = {r2_train:.3f})')

subplot(1, 2, 2)
scatter(y_val, y_pred_val, alpha=0.5, s=10)
plot([y_val.min(), y_val.max()], [y_val.min(), y_val.max()], 'r--', lw=1)
xlabel('Actual Impurity')
ylabel('Predicted Impurity')
title(f'Validation (R² = {r2_val:.3f})')

tight_layout()
show()

```



In [14]: *# Analyze how R^2 changes as a function of Cp-selected step (model complexity)*

```

r2_train_list = []
r2_val_list = []
num_vars = []

for step in range(path_train.shape[1]):
    coef_step = path_train[:, step]
    n_active = np.sum(coef_step != 0)
    if n_active == 0:
        # Skip degenerate model (no variables)
        r2_train_list.append(np.nan)
        r2_val_list.append(np.nan)
        num_vars.append(0)
        continue

    # Training predictions
    y_pred_train = softplus(z_bar_train + X_s_train @ coef_step)
    r2_train_list.append(r2_score(y_train, y_pred_train))

    # Validation predictions
    y_pred_val = softplus(z_bar_train + X_s_val @ coef_step)
    r2_val_list.append(r2_score(y_val, y_pred_val))
    num_vars.append(n_active)

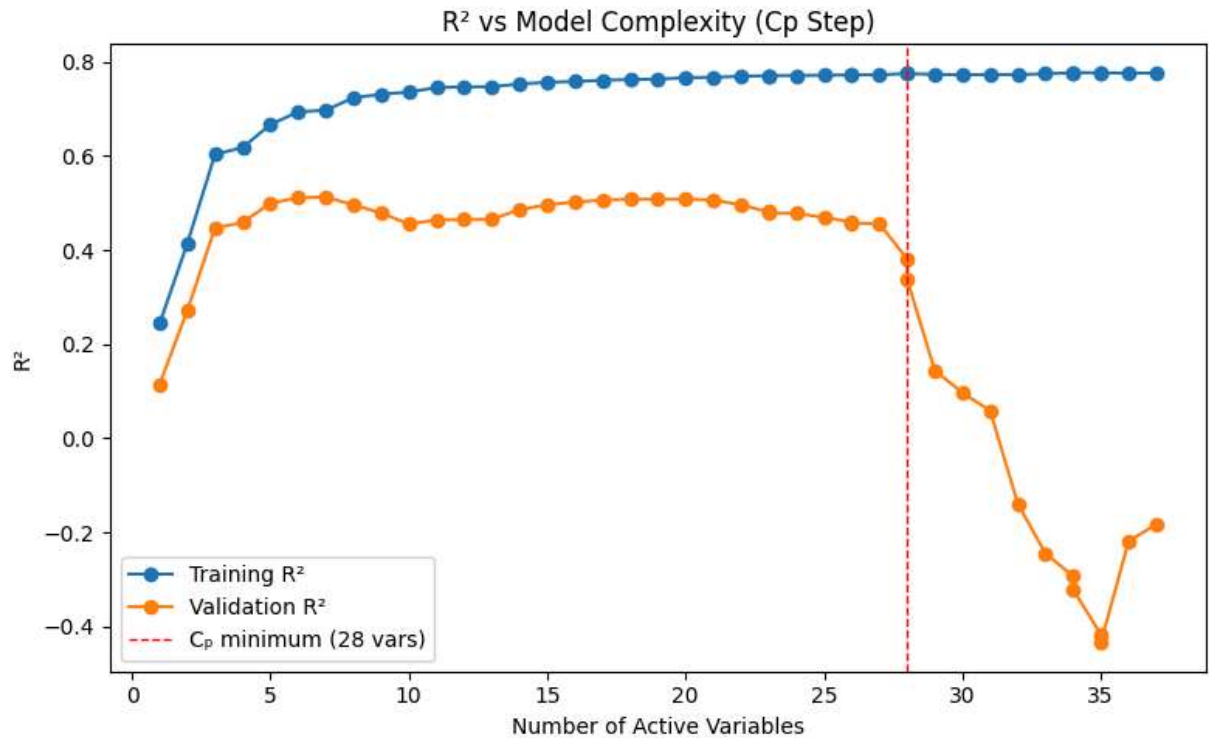
figure(figsize=(8, 5))

```

```

plot(num_vars, r2_train_list, label='Training R²', marker='o')
plot(num_vars, r2_val_list, label='Validation R²', marker='o')
axvline(num_vars[best_step], color='r', ls='--', lw=1, label=f'Cp minimum ({num_vars[best_step]} vars)')
xlabel('Number of Active Variables')
ylabel('R²')
title('R² vs Model Complexity (Cp Step)')
legend()
tight_layout()
show()

```



```

In [16]: # Refit with the optimal number of variables and save
import joblib

best_n_vars = int(p_s[best_step]) - 1 # e.g. 10

pipe_best = Pipeline([
    ('scaler', StandardScaler()),
    ('lars', Lars(n_nonzero_coefs=best_n_vars))
])
model_best = TransformedTargetRegressor(
    regressor=pipe_best,
    func=softplus_inv,
    inverse_func=softplus
)
model_best.fit(X_train, y_train)

joblib.dump(model_best, 'lars_dow_A10.pkl')
print("Model saved.")

```

Model saved.

HW answer for question 2:

My model massively underperforms compared to Qin et al., which is disappointing. Based on the training and validation R^2 values, I am significantly overfitting my system; not surprising, since I am using many more variables than Qin et al. did. I think going back and optimizing the variable number selection would improve this, but I am still behind on the training set R^2 value as well. Compared to the PLS, LARS performs much better, which is another deviation from Qin et al. who reported similar and complementary performance from both systems. One of the primary issues I run into is the false prediction of impurity spikes in the validation prediction, a phenomena which may be mitigated by OBL.